RAPPORT DE PROJET

Simulateur de Microprocesseur Motorola 6809

Filière : LST GI

Année universitaire : 2025/2026

Module : Architecture des ordinateurs

1. Introduction Générale

Le microprocesseur Motorola 6809, apparu à la fin des années 1970, est reconnu comme l'un des processeurs 8 bits les plus avancés de son époque. Il se distingue par une architecture orthogonale, un jeu d'instructions riche et une séparation claire entre données et adresses.

Ce projet vise à développer un simulateur logiciel du Motorola 6809, écrit en Java, permettant :

- L'apprentissage du langage assembleur 6809
- La compréhension du fonctionnement interne du processeur
- Le test et le débogage de programmes assembleur
- La visualisation graphique de l'état du processeur et de la mémoire

2. Architecture Globale du Système

2.1 Diagramme Conceptuel

```
+------------------+
|      Main        |
+------------------+
+------------------+
|  SimulatorGUI    |
+------------------+
+------+    +--------+
| CPU  |    | Memory |
+------+    +--------+
```

2.2 Description des Modules

Module	Rôle
Main	Démarrage de l'application
SimulatorGUI	Interface graphique, assembleur, débogueur
Cpu6809	Émulation du processeur
Memory	Gestion mémoire RAM / ROM

3. Émulation du Processeur – Classe Cpu6809

3.1 Rôle et Responsabilités

La classe Cpu6809 représente le cœur du simulateur.

Elle est responsable de :

- La gestion des registres internes
- Le décodage des instructions
- L'exécution instruction par instruction
- La mise à jour des drapeaux
- Le suivi de l'instruction courante

3.2 Registres du Processeur

Registres 16 bits

Registre	Description
PC	Compteur ordinal
S	Pile système
U	Pile utilisateur
X	Registre index
Y	Registre index

Registres 8 bits

Registre	Description
A	Accumulateur A
B	Accumulateur B
DP	Direct Page
CC	Code Condition

3.3 Code Condition (CC)

Les drapeaux suivants sont implémentés :

Bit	Nom	Rôle
N	Négatif	S'active si le résultat est négatif (bit 7 à 1)
Z	Zéro	S'active si le résultat d'une opération est nul

3.4 Initialisation du Processeur

La méthode reset() :

- Initialise le PC à 0xF000
- Réinitialise tous les registres
- Efface les drapeaux

4. Cycle d'Exécution du CPU

4.1 Principe Général Chaque appel à la méthode : `cpu.step();` représente un cycle processeur.

4.2 Algorithme du Cycle

1. Lire l'opcode à l'adresse PC
2. Incrémenter le PC
3. Décoder l'opcode via un switch
4. Lire les opérandes si nécessaire
5. Exécuter l'instruction
6. Mettre à jour les drapeaux
7. Mettre à jour l'instruction courante

4.3 Exemple : Instruction LDA #imm

```
LDA #$10
```

Étapes internes :

- Lecture opcode 0x86
- Lecture immédiate de la valeur 0x10
- Chargement dans A
- Mise à jour du drapeau Z et N

5. Instructions Implémenté

5.1 Instructions de Chargement

- LDA, LDB
- LDX, LDS

5.2 Instructions de Stockage

- STA, STB

5.3 Instructions Arithmétiques

- ADDA, ADDB
- INCA, INCB
- DECA, DECB

5.4 Instructions de Comparaison

- CMPX

5.5 Instructions de Branchement

- BRA (inconditionnel)
- BEQ (si Z = 1)

- BNE (si Z = 0)
- JMP

5.6 Instructions Spéciales

- EXG A,DP
- TFR A,DP
- PSHS, PULS
- CLRA, CLRB
- NOP
- END / SWI

6. Gestion de la Mémoire – Classe Memory

6.1 Organisation Mémoire

Plage	Type	Initialisation
0000–7FFF	RAM	00
8000–FFFF	ROM	FF

6.2 Accès Mémoire

- Lecture toujours autorisée
- Écriture autorisée sur toute la mémoire (simplification pédagogique)

7. Assembleur Intégré

7.1 Fonctionnement Général

L'assembleur intégré traduit un langage assembleur simplifié directement en opcodes machine.

7.2 Algorithme d’Assemblage

Étapes de l'algorithme :

1. Prétraitement : Le code source est découpé ligne par ligne et converti en majuscules. Les lignes vides sont ignorées.

2. Analyse Syntaxique : Chaque ligne est séparée en un mnémonique (ex: LDA) et un opérande (ex: #$10).

3. Détection du Mode d'Adressage :

• # : Immédiat.

• < : Direct.

• ,X : Indexé.

4. Génération du Bytecode :

• Recherche de l'équivalent hexadécimal de l'instruction (Opcode).

• Conversion de l'opérande hexadécimal en valeur numérique.

• Écriture séquentielle dans la mémoire à partir de l'adresse de base $F000.

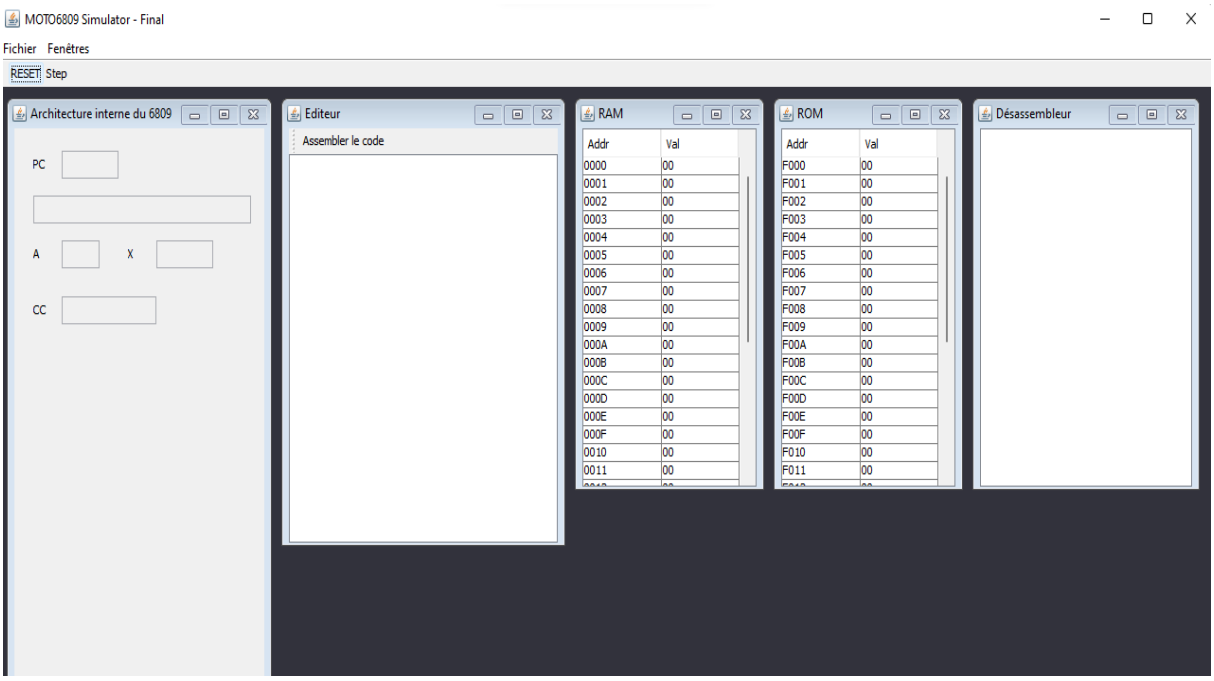
5. Validation : Une vérification stricte rejette les symboles invalides (ex: $#) pour garantir la robustesse du programme émulé.

7.3 Validation Syntaxique

Exemples d’erreurs détectées :

- Absence de # pour immédiat
- Mauvais symbole ($#, #<)
- Valeur hexadécimale invalide
- Mnémotechnique inconnu

8. Interface Graphique et Débogueur



8.1 Fenêtres Principales

L'interface est composée de plusieurs fenêtres flottantes :

- Architecture interne : Affiche l'état des registres et l'unité arithmétique et logique (UAL) en temps réel.
- Éditeur : Zone de saisie pour écrire votre code assembleur 6809.
- RAM/ROM : Tableaux permettant de surveiller le contenu de la mémoire.
- Programme : Affiche le code machine désassemblé après compilation.

8.2 Débogueur Pas à Pas

Fonctionnalités :

- RESET du processeur
- Exécution instruction par instruction
- Mise à jour automatique :
  - Registres
  - Mémoire
  - Instruction courante

9. Guide d’Utilisation

Comment exécuter un programme ?

1. Saisie du code : Écrivez vos instructions dans la fenêtre Editeur.

2. Assemblage : Cliquez sur le bouton "Mise à jour" dans la fenêtre Editeur. Si le code est correct, un message "Assemblage OK" apparaîtra.

3. Exécution :

- Step : Cliquez sur le bouton "Step" en haut de l'écran pour exécuter les instructions une par une. Observez les registres changer de couleur bleue à chaque étape.

- Reset : Cliquez sur "RESET" pour remettre le processeur à son état initial (PC à $F000).

Signification des drapeaux (CC)

9. Conclusion Générale

Ce simulateur du Motorola 6809 constitue une plateforme pédagogique robuste, permettant :

- La compréhension de l’architecture interne
- L’apprentissage du langage assembleur
- Le débogage précis pas à pas
- Une visualisation claire et intuitive du fonctionnement du processeur